

课程笔记

[LLM Agents SP25] Inference-Time Techniques for LLM Reasoning —Xinyun Chen

基于 Xinyun Chen 授课内容整理

2025-01-27



视频作者/频道: Berkeley RDI

发布日期: 2025-01-27

视频时长: 1:21:32

视频链接: <https://www.youtube.com/watch?v=g0Dwtf3BH-0>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言	3
1.1 本章小结	3
2 推理时提示与令牌预算	3
2.1 扩大 token 预算以暴露推理链	3
2.2 提示设计的约束	4
2.3 成本可观测性与预算控制	4
2.4 本章小结	4
3 推理时计算调度	4
3.1 Token/Width/Iteration 三角调度	4
3.2 成本与延迟权衡	5
3.3 推理时算力分层调度	6
3.4 Latency-Controlled Prompting	7
3.5 本章小结	7
4 搜索与多候选选择	8
4.1 自一致采样与多候选	8
4.2 Self-Consistency 与 Search 的协同	8
4.3 验证器与排序基线	9
4.4 搜索的工程线索	9
4.5 本章小结	9
5 验证器设计深入	9
5.1 Stepwise verifier 的构成	9
5.2 Verifier gating loops	10
5.3 Multi-verifier orchestration	10
5.4 本章小结	11
6 树状探索与树级评分	11
6.1 Tree-of-Thought 与分支搜索	11
6.2 树状控制循环与预算自适应	12
6.3 本章小结	12
7 迭代自我改进与反馈	12
7.1 Trace Feedback 与步级评分	12
7.2 自我修正提示与风险	13
7.3 本章小结	14
8 策略融合与调度	14
8.1 组合提示、搜索与 trace	14
8.2 预算调度与指标	14
8.3 本章小结	14

9 Benchmark 解读与研究方向	14
9.1 Arc AGI / O3 曲线解读	14
9.2 Benchmark 仪表与预算分层	15
9.3 开放研究问题	15
9.4 本章小结	16
10 部署与观测	16
10.1 监控 inference-time 指标	16
10.2 Budget SLO 与警报	17
10.3 本章小结	17
11 实践示例与指令模板	17
11.1 Prompt Template 演练	17
11.2 Self-Consistency Orchestration	18
11.3 本章小结	18
12 风险与治理	18
12.1 风险识别	18
12.2 治理与演练	18
12.3 本章小结	19
13 案例分析与下一步	19
13.1 Arc AGI Puzzle Walkthrough	19
13.2 高价值任务的推理时部署	19
13.3 本章小结	20
14 总结与延伸	20
14.1 本章小结	20
14.2 进一步阅读	21

1 引言

本讲一开场便由 Xinyun Chen 提到 “2025 will be the year of Agents”，并说明本学期 Berkeley RDI 的 Advanced LLM Agents SP25 课程在 Google DeepMind 与 Cal 的协作下，既有应用 track 的成熟案例，也新增着眼于 inference-time reasoning 的研究 track。讲师强调 15,000+ 名 MOOC 报名者、3,000+ 名 hackathon 参与者以及超过 20 万美元的行业资源，意在表明本课程不是纯理论演示，而是真正要把推理时技术部署到复杂工程场景中。

课程定位

模块化地把推理时技术拆成“提示 + 搜索 + 迭代”三部分，讲师将在本讲详细分享每一环的策略、指令模板与工程要点，让我们在“推理时投入算力”这个主题下同时把 token/width/depth 三个维度照顾到。

1.1 本章小结

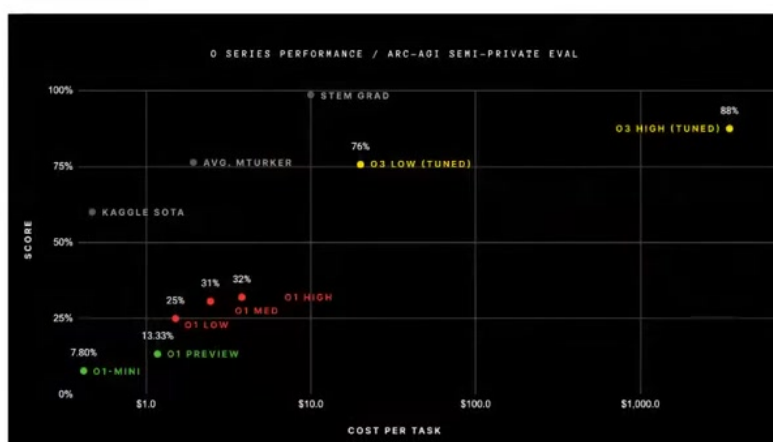
引言为本讲设定了 Agent 冷启动背景，强调 inference-time compute 作为不改权重即提升推理的杠杆，并把内容组织成提示、搜索与迭代三段，后续章节按照这一逻辑展开。

2 推理时提示与令牌预算

2.1 扩大 token 预算以暴露推理链

Chen 以 Arc AGI puzzle 为例展示推理时 token 预算的曲线：传统模型在 budget 约 20 美元时止步 25% 以下，而 OpenAI O3 在设定 20 1000 美元预算、配合示例提示与执行 trace 的前提下，可把准确率推到 87.5%。此处幻灯片（图 1）清楚展示推理精度随 token 预算拉长腰部的非线性趋势。

Performance improves with more inference-time compute



O3 achieved 87.5% accuracy on ARC-AGI, with >\$1k test-time cost to solve each task

- Other existing LLMs achieved <25% accuracy if no special inference-time techniques were used

<https://arcprize.org/blog/oai-o3-pub-breakthrough>

Powered by Zoom

图 1: Arc AGI 的推理时曲线与 OpenAI O3 的 token-cost 布局。¹

¹视频画面时间区间：00:08:56–00:10:20，使用 O3 曲线说明 token 扩展可达到 human-level accuracy。

推理时曲线的三重启示

1) 当任务复杂度升高时，token 预算需要从几十推到上千才能看到显著提升；2) 增加预算必须配合示例提示与 stepwise scoring，否则仅靠更长的 prompt 无法抬升准确率；3) 高预算模式适用于竞赛级别或验证任务，不适合原生低延迟对话。

2.2 提示设计的约束

提示需要两条能力：一是鼓励模型生成长 Chain-of-Thought，让多轮合理推理序列自然浮现；二是将任务难度 signal 内置在 prompt 中，使模型自动调节生成深度。Chen 在 44 分 31 秒明确指出，“提示只能整出更长推理的前提是，被提示的模型知道自己面对的是难题”。

提示设计双标准

1. 长 CoT：使用分步模板、问答对或多例示范，让模型持续追踪当前状态；2. 难度标记：把 problem difficulty、context length、expected depth 显示在 prompt 中，让模型在 inference-time 自适应调整 depth。

2.3 成本可观测性与预算控制

扩展 token 预算要参考真实成本：从 1 美元、20 美元到 1,000 美元每题的区别，不只是钱，还带来 latency 与算力消耗。陈列的现实风险在于“高预算让任务不可解释，也可能吞噬微服务资源”。

预算跑飞的风险

一旦推理时预算拉到上千美元，就必须限制在高价值子任务，否则会让调用频率踏空、系统吞吐率下降甚至引发平衡失控。建议提前设定 sampling 缓存策略，把最昂贵的 search 留给 verified-critical path。

2.4 本章小结

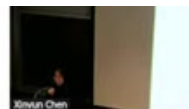
推理时提示需要在“鼓励长 CoT”与“任务难度感知”之间找到平衡，同时把 token 预算、成本与 latency 指标纳入监控。Arc AGI 曲线图提醒我们，token 数量扩展有效但成本逐步饱和，必须让提示、验证与预算协调运作。

3 推理时计算调度

3.1 Token/Width/Iteration 三角调度

Chen 在 15 分 32 秒处用一张幻灯片将“提示 → 搜索 → 迭代”画成一个 token / width / iteration 的三角形，强调不能只加 token，还要安排多候选搜索和多轮自我修正（图 2）。工程上常见的做法是先用 prompt 让模型生成更长的候选，然后控制每个 step 的 branching，再为每个 candidate 预留一到两轮的 feedback loop。

Outline



- Part 1: Introduction to basic prompting techniques
 - Use **more token budget** to generate **a single solution**
- Part 2: Search and selection from multiple candidates

Powered by Gemini

图 2: 推理时计算的三阶段 pipeline: prompt $\rightarrow$ search $\rightarrow$ iteration。²

三角调度的工程量化

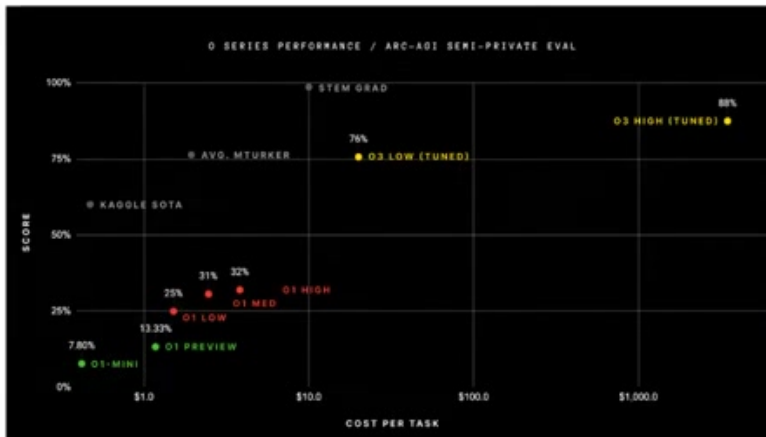
1. Token: 测量一次完成 response 所需的总 token budget;
2. Width: 记录采样生成候选的数量与 diversity;
3. Iteration: 评估自我改进轮数 (如 trace feedback / self-correction loops) 对 accuracy 的边际收益。

3.2 成本与延迟权衡

Chen 反复强调在可观测指标上追踪 inference-time 消耗: 一条 1,000 美元的算力曲线固然令人印象深刻, 但很多实际部署场景只能接受 20 50 美元的 token 预算。图 3 来自 00:10 的讲解, 指出成本和 latency 之间存在明显的 tradeoff, 提示在低延迟通道应优先在 search 与 verifier 上节省, 而在高价值任务中可以放开 token 预算。

²视频画面时间区间: 00:15:10–00:16:10, 展示 token-width-iteration 的调度曲线。

Performance improves with more inference-time compute



O3 achieved 87.5% accuracy on ARC-AGI, with >\$1k test-time cost to solve each task

- Other existing LLMs achieved <25% accuracy if no special inference-time techniques were used

<https://arxiv.org/abs/2408.01587>

Reproduced by Chen

图 3: 推理时成本与延迟的平衡选项。³

不可控的预算膨胀

在没有 verifier 或搜索控制的情况下，仅靠放宽 token 预算容易造成“预算膨胀”，成本飙升且 accuracy 没有同步提升。建议用缓存、采样池与 verifier 阶段把最昂贵的候选限定在已知 high-value 任务上。

3.3 推理时算力分层调度

推理时算力必须在 prompt/sampling、search、iteration 三个阶段之间划分清晰的预算窗口。Chen 现场提到的做法是先用 task difficulty 信号设置一个总 token cap（例如 400 600 token），然后根据 search depth 与 iteration 目标倒推每个阶段的 share：prompt 预留 25 t 与 100 token 之间，search 阶段按 branch 数加宽，iteration 轮次每轮占 80 120 token，同时预留 20 t 作为 verifier scoring 的余量。这个分层机制让我们在 budget 不变的情况下，通过不同阶段的预算 reallocation 达到偏重 accuracy 或 latency 的不同策略。

³视频画面时间区间：00:09:40–00:10:30，展示 20-1,000 显著不同的预算阶梯。

表 1: 推理时算力分层示例

阶段	关键指标	典型预算	工程提示
Prompt / Sampling	Input tokens + example CoT	25% 35% of total	把 difficulty metadata 与 instrumentation hints embed 进 prompt, 保证 sampling 能在 high temperature 下启动
Search (branching)	Width、stepwise score、branching ratio	40% 50%	用 verifier gating 控制 depth, 让每次展开都能带来 score margin 准备
Iteration / Feedback	Trace iterations、consistency votes	20% 30%	迭代阶段常见 self-correction 或 verifier replay, 占用 token 但能显著修正 mistakes
Verifier / Logging	Score margin、drift detect	10% 15%	并行埋点, 如 sampler metrics、verifier distribution, 保证 latency tuned

算力分层的控制回路

1. 先设置 token cap + latency target; 2. 按 stage 粗略分配比例并写入 dashboard; 3. 观察运行中每条 query 的 stage 占比, 若 search/iteration drift 过高就用 prompt dial-back 重新收口。

3.4 Latency-Controlled Prompting

在 Operations 级别, 为了避免 latency 脱轨, Chen 建议把 prompt 调整为 “budget-aware templates”: 在 prompt 中注明 “预算越过 400 token 则回退” 或 “请在 1 2 个 iteration 内完成推理”。同时, 我们可以在 prompt 里插入 instrumentation hint (如 ‘Sampling policy: alt-branch’), 让 dispatch 系统在 sampling 阶段就限制候选数。这些 micro-guidelines 有助于把 cost/latency tradeoff 直接暴露在 inference-time template 而非 after-the-fact 的监控警报。

Latency guard 设计要点

Prompt 中的 guard rail 要具体: 写清楚 “最多 2 个 iteration” 比简单说 “控制 latency” 更有效; 还可以用 ‘If token > 400 then reply "budget exceeded"' 作为硬性限制, 迫使模型提前汇报 token usage。

3.5 本章小结

推理时计算调度要求同时衡量 token 预算、candidate width、iteration depth, 并把 latency guard 与多阶段算力分层纳入初始 budgeting。把这三者视为工程的三条轴线, 才能在不同应用场景下实现既可靠又高效的推理。

4 搜索与多候选选择

4.1 自一致采样与多候选

在 search 层面，自一致性（Self-Consistency）成为核心武器：允许模型生成多条候选，再用多数票或 consistency score 挑出最稳定的 final answer。如图 4 所见，随着候选数量从 10 织到 40，accuracy 仍在提升，远甩基于 log-prob 的排序。

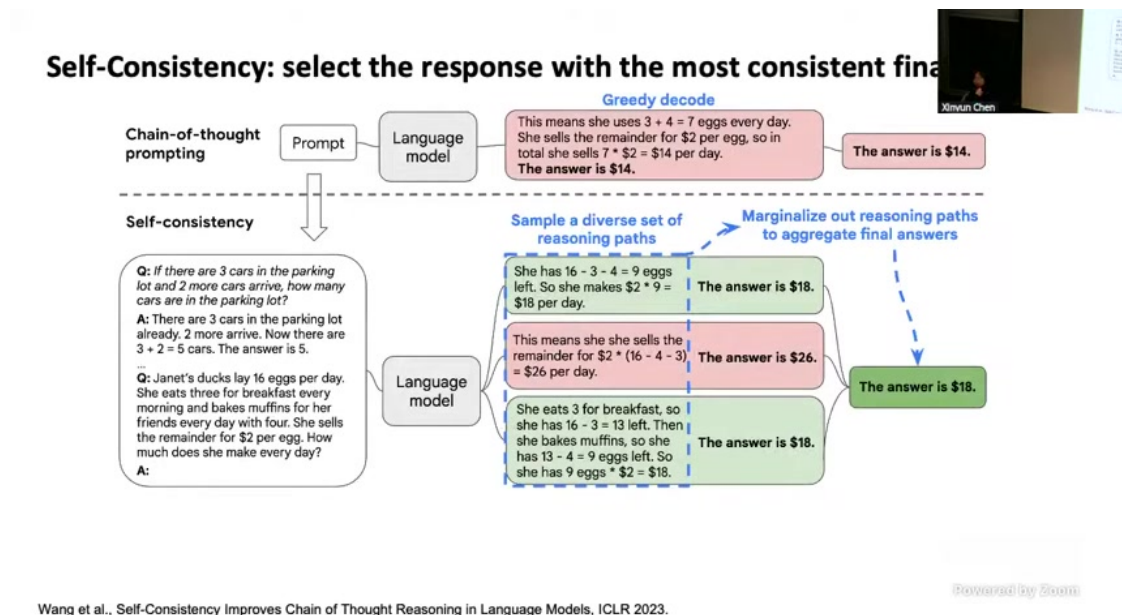


图 4: Self-Consistency 曲线对比 log-prob 排序与其他 sampling 策略。⁴

多候选采样三要素

1. 多样性: 高 temperature、nucleus sampling 与 prompt 顺序扰动保证 candidate 之间差异;
2. 选择策略: 多数票优于 log-prob，除非为某任务训练了强 verifier;
3. 资源分配: multi-sample 仅在需要更高准确率、允许延迟的场景打开。

4.2 Self-Consistency 与 Search 的协同

Self-Consistency 为 search 提供了更宽的 width，而 beam search/Tree-of-Thought 通过 stepwise scoring 控制 depth。Chen 在 47 分 02 秒指出，理想的流程是先用 sampling 生成 20 40 个 candidate，再用 search + verifier 判断哪条路径可继续深化，最后用 consistency 投票稳定最终 answer。简单使用 majority vote 虽然可靠，但要在 inference-time 控制成本，仍需和 search 彼此配合，一边确保候选多样，一边用 verifier 在 early stage 就截断 bad paths。

不要只靠其中一个模块

如果只运行 Self-Consistency 而不加任何 search，可能会在 token 成本上耗尽却继续重复相似错误；反之，如果只跑 search 而不加 sampling，多数时候只能走到局部最优。因此必须在 inference-time pipeline 中把 sample 生成、search 拓展、consistency 投票三者串联。

⁴视频画面时间区间：00:45:05–00:50:05，图中多样化采样与投票显著优于单一 log-prob 排序。

4.3 验证器与排序基线

若训练出 stepwise verifier，可在生成早期筛掉低质分支——DeepMind 的 AlphaCode trace 就是一个例子，它自述 code execution 并在每一步给分，进而影响后续 search。标量 verifier 要兼顾稳定性与可解释性，否则可能把复杂正确答案误判为“不可解释的黑盒”。

弱 verifier 的陷阱

若 verifier 过度偏向短输出或训练集模板，它会把多步骤推理判定为“低分”而舍弃，反而让 self-consistency 收敛到一组常见但错误的解。要让 verifier 同时敏感 accuracy 与 reasoning depth。

4.4 搜索的工程线索

在 52 分 50 秒的幻灯片中，Chen 进一步把 sampling、search、verifier 的输出量化：采样 count、candidate score、voter margin 等指标要在 inference-time logs 中持续扫描。一个常见 pipeline 是用 high-temperature sampling 生成 30 条 response，用 stepwise verifier 给每条 response 赋分，再用 majority vote 冠军最后解。每个环节都应有对应的监控表格，以便在真实部署中抓到“今天采样 diversity 缺失”或“verifier drift”的 warning。

搜索流水线指标

1. Sampling depth：候选数量与 diversity；
2. Verifier quality：stepwise score 趋势与稳定性；
3. Voting stability：majority margin 或 consistency rate，用于判断是否需要额外 search。

4.5 本章小结

自一致性配合多样化采样可以让模型从 mistakes 中恢复，但要结合高质量 verifier 排序。整套机制需要在 accuracy、token budget 与 latency 之间做权衡。

5 验证器设计深入

5.1 Stepwise verifier 的构成

高质量 verifier 至少由三个子模块组成：step scoring、trace evaluation 与 metadata gating。Chen 强调不要把 verifier 当成 post-hoc 的黑盒评分器，而要把它写进 sampling/search pipeline。例如在 Trace 自生成的分支上，可以让 verifier 先打 depth score（chain-of-thought 的长度与复杂度）、再打 validity score（是否符合 problem constraint）、最后再打 confidence margin（多数投票 margin）。这三个指标构成了 stepwise verifier 的输出 tuple，在 search loop 中用作 pruning gate。

表 2: Stepwise verifier 输出

指标	典型计算方式	作用
Depth score	Prompt override / reasoning depth heuristic	避免 verifier 过度偏好短 output
Validity score	Constraint check + domain rules	保证 branch 不偏离 template
Confidence margin	Consistency across candidates	为 majority vote 提供 margin base

Verifier 训练策略

1. 用 supervised trace 数据训练 validity + depth scorer; 2. 在 inference-time 里把 control token '@validator' 赋予 higher probability; 3. 结合 hashing checkpoints (如 trace hash) 避免 verifier 在重复 steps 时 drift。

5.2 Verifier gating loops

Verifier 需要和 search 形成闭环: 每当 candidate 被打开判断后, verifier 会给出 margin, 并把 margin 作为 branching priority。Chen 在 36 分 10 秒展示的 gating loop 是: candidate → verifier → score queue → branch expansion/termination。这个过程可视为一个多队列 scheduler, 其中 margin 越高的 branch 会被平行推入 tree, 而 margin 低的会被 early stop。若某阶段 margin 全局下降, 就说明 sampling 的 diversity 可能 collapse, 需要立即回退到 “prompt plus trace” template 并触发 ‘trace replay’。

Margin collapse 风险

当 verifier margin 全局下滑时, 不要立刻加 token, 而是先回头看 sampling diversity 与 prompt difficulty。低 margin 可能是 dataset shift, 也可能是 prompt drift, 先跑 ‘trace replay’ 看是 self-correction drift 还是 prompt drift, 再决定是否扩展 search。

5.3 Multi-verifier orchestration

实践中会同时部署多个 verifier: 一个快速的 heuristic scorer (例如 tf-idf + regex), 一个基于 LM 的 trace evaluator, 以及一个 deterministic checker (如 proof verifier)。在 inference-time pipeline 里把这三层串联, 让 fast verifier 先快速 trim, 然后把 high-scoring branch 交给 slow-but-robust verifier 重评。Chen 强调 audit log 必须记录每层 verifier 的 decisions, 这样才能在 anomaly 时定位 “哪一层把 candidate 跳掉了”。

多 verifier 的协同节奏

1. Fast verifier: 低 cost, 用于 early pruning; 2. LM verifier: 中等 latency, 负责 reasoning depth; 3. Deterministic checker: 高成本但高信度, 常用于 high-stakes query 结束时; 4. 把每层 output 写在 dashboard 里, observability 团队就能看出是哪一层造成 margin drift。

5.4 本章小结

Verifier 设计不仅需要高质量 scoring 机制，还要和 search 形成 budgeting gating，甚至通过多 verifier orchestration 保护 high-value queries。只有把 verifier 变成 inference-time control loop，才能避免 budget race 或 drift 导致 reasoning 崩盘。

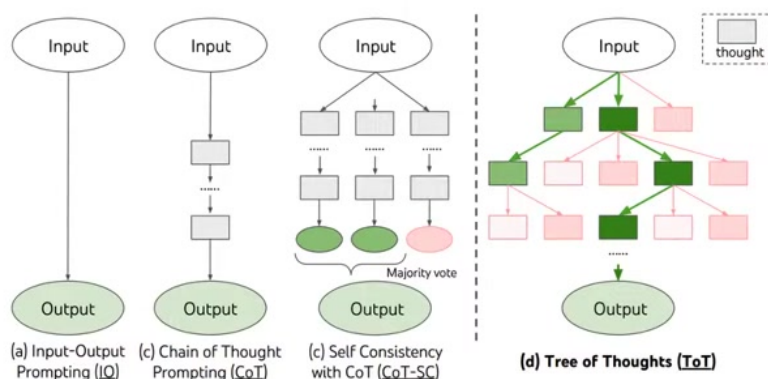
6 树状探索与树级评分

6.1 Tree-of-Thought 与分支搜索

Tree-of-Thought 允许在每一步展开多条分支，使用 stepwise scoring 判断哪些 branch 值得展开。如图 5 所示，beam search 只保留 top-k whereas Tree-of-Thought 可以在 step-level 就过滤 promisingness。

So far: response selection only after the full responses are generated

- This does not fully utilize a step-wise scorer!
- LLM + tree search: prioritize the exploration of more promising partial solutions



Yao et al., Tree of Thoughts: Deliberate Problem Solving with Large Language Models, NeurIPS 2023.

Powered by Zoom

图 5: Tree-of-Thought 与 beam search、自一致性的对比示意。⁵

树状搜索的三个实践洞察

1. 不是所有分支都要展开，优先级排序可由 step verifier 直接给分；
2. step-level scoring 可以在 branch 还未完成前淘汰低质路径，因此减小 token 成本；
3. beam search 可与 sampling 组合：先用 sampling 出 diverse seed，再用 beam/tracing 筛选 promising ones。

树状控制的操作逻辑

在 Tree-of-Thought 中，控制 loop 常用 “branch proposal → step scoring → prune” 三步：第一步是 warm-start 多个 branch，第二步用 verifier 给每个 branch 分数，第三步根据 budget 剪枝。Chen 建议在 training log 中记录每个 branch 的 token count 与 score，以便找到 “哪个 step 的 score drift 最明显”。

⁵ 视频画面时间区间：01:01:24–01:04:30，展示 step evaluation 在树状搜索中的位置。

树状 branching 不可无限扩展

尽管 tree search 能在 early stage 提高 accuracy, 但如果没有清晰的 pruning 策略, branch 数量会指数级增长。务必设定 token threshold (如最多探索 3 层)、引入 stepwise verifier gating, 并用 logs 监控 branching ratio。

6.2 树状控制循环与预算自适应

在 Tree-of-Thought pipeline 中, 必须有一个控制循环来决定什么时候拓展、什么时候 prune、什么时候 roll back。Chen 现场给出的控制逻辑是: 每轮 step 结束后检查 ‘branch score / step token’ 是否超过 threshold; 若低于 threshold, 则立即把该 branch push 到 aborted queue 并把剩余 token 回收 to global budget。这个 loop 通过 SLO instrumentation (如 ‘branching_ratio’ 与 ‘score_per_token’) 曝光出 tree search 中那些掠过的无效分支。

表 3: Tree search 控制指标

指标	监控意义	Pipeline action
Branch score	判断分支 promisingness	Score < 0.2 时 prune 并 trigger ‘trace replay’
Branching ratio	观察 active branches 与 token 关系	> 28% 时把 new branches limit 到 2
Score per token	控制 depth 与 cost	下滑表示 search drift, 需调低 temperature 或 early stop

Tree 控制 loop 的经验

1. 设定 ‘early score check’, 即在 branch 未完成前便计算 partial score; 2. 用 per-branch token cap 强制分支不能无限制延伸; 3. 在 control loop 异常 (例如 branching ratio 突破) 时, 马上触发 conservative mode (降低 prompt temperature + 限制 iteration)。

6.3 本章小结

Tree-of-Thought 等树状结构在 inference-time search 中填补 beam search 与 sampling 之间的 gap, 只要配上健全的 step verifier 和控制循环, 就能在有限 budget 下实现 high-quality breadth exploration。

7 迭代自我改进与反馈

7.1 Trace Feedback 与步级评分

Chen 介绍 AlphaCode 的 trace feedback, 将模型执行轨迹用自然语言 reify, 形成 stepwise verifier。通过让模型自生成 line-by-line debug, 它能识别哪些中间状态 promising, 并用 text-level 评估影响后续 search。

Self-debugging results

(a) Results on the Spider development set.

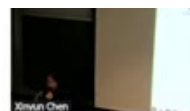
Spider	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	81.3	71.1	73.2	64.7
Simple	81.3	72.2	73.4	64.9
+Expl.	84.1	72.2	73.6	64.9

(b) Results on TransCoder.

TransCoder	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	80.4	89.1	77.3	70.0
Simple	89.3	91.6	80.9	72.9
UT	91.6	92.7	88.8	76.4
+ Expl.	92.5	92.7	90.4	76.6
+ Trace.	87.9	92.3	89.5	73.6

(c) Results on MBPP.

MBPP	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	61.4	67.6	72.8	47.2
Simple	68.2	70.8	78.8	50.6
UT	69.4	72.2	80.6	52.2
+ Expl.	69.8	74.2	80.4	52.2
+ Trace.	70.8	72.8	80.2	53.2



Powered by Beamer

图 6: Trace feedback 示例画面，展示 line-by-line explanation 的可视化。⁶

Trace feedback 的高阶价值

1. 把 step evaluation 转化为 natural language trace, 使模型自己成为 verifier; 2. AlphaCode 通过 trace 观察 context renovation, 从而在 code generation 环节捕捉 subtle bugs; 3. 更丰富的 trace 可直接作为 RL 奖励 signal, 为 AlphaProof 与 AlphaGeometry 等系统提供 inference-time 改进路径。

7.2 自我修正提示与风险

当没有 oracle verifier 时, 自我修正往往把错误转换成另一种错误, Chen 以 01:13:23 处的负例提醒大家。设计 general-purpose feedback prompt 时, 需突出 consistency、加入 trace context、并设定“不要翻动已确认部分”的 guard rails。

自我修正提示调参方向

1. 强调 consistency: 提示模型“列出两个可能的新步骤”, 而非像“检查答案”这样模糊的引导; 2. 注入 trace context: 将 line-by-line explanation 再次作为 prompt, 帮助模型用自身 reasoning 判断 correctness; 3. 控制跳跃: 增加“保留已确认推理”语句, 防止模型因 feedback drift 把已验证的部分推翻。

无 oracle 的 self-correction 坏处

当只有 general feedback、没有 ground truth, 模型会因为看不清哪条 response 可靠, 反而保守地改写正确答案或放弃多步骤 reasoning, 导致 accuracy 反而下降。

⁶视频画面时间区间: 01:11:20–01:12:15, 讲解 trace feedback 的运作方式。

7.3 本章小结

Trace feedback 与 self-correction 需要 reliable verifier 与 prompt guard rails, 否则模型会在多个 round 后退步。设计 feedback 时必须兼顾 consistency 引导、trace context 和可解释性限制。

8 策略融合与调度

8.1 组合提示、搜索与 trace

Chen 强调上述策略不是孤立: 可以先用 Tree-of-Thought 拓展 branch, 再通过 self-consistency 投票稳定 final answer, 最后让 trace feedback 迭代验证。换句话说, pipeline 形如 prompt $\rightarrow$ search $\rightarrow$ iteration, 每一环都有对应的指标与 budget。

8.2 预算调度与指标

实际部署需对 token、branch、iteration 三重预算做调度。复杂任务可在前期分配更多 token, 在 search 阶段用 verifier 控制 branch 数量, 在 iteration 阶段限制 feedback rounds。指标上要同时记录 accuracy、token count、latency。

预算调度原则

1. 自动伸缩: 用 task difficulty signal 调整 token/branch depth; 2. 优先级队列: 借助 verifier 给每个 branch 打分, 控制展开顺序; 3. 观测表征: 记录每个模块 (提示、采样、trace) 在 deployment 中的 F1、token count 与 latency, 以便随时重新权衡资源。

策略融合的实践提示

不要把 prompt、search、trace 视为独立系统, 而是把它们当成一个预算链条: 当 search depth 增长时, 自动扣减 token budget 或限制 iteration; 当 iteration 深度增加时, 把 search branching 退回到安全值。这样才能在部署中避免“所有模块独立延长, 预算指数级膨胀”的现象。

8.3 本章小结

策略融合意味着把提示、搜索、trace 看作一条闭环 pipeline。通过预算调度、优先级队列与指标回路, 可以在 inference-time compute 下保持高准确率与可解释性。

9 Benchmark 解读与研究方向

9.1 Arc AGI / O3 曲线解读

Arc AGI 作为 reasoning benchmark 的核心指标, 最早能让 1,000 美元预算达到 87.5% accuracy, 而 20 美元以下只能维持在 25%。Chen 指出这条曲线并非只是“给钱就能做”, 而是要同步排布 sampler、search 和 verifier, 才能让每一笔预算都转化为有效 accuracy。OpenAI O3、AlphaProof/AlphaGeometry 等 demo 的实践, 牢牢依赖 inference-time compute, 而不是训练域。

表 4: 主要 Benchmark 的推理时预算/性能对比

Benchmark	Token / Cost budget	Observed behavior
Arc AGI	\$20 \$1,000 per query	25% $\rightarrow$ 87.5% accuracy; the curve saturates after \$500 without better search/verifier
OpenAI O3	\$20 \$200	Consistency + trace feedback gives human-level accuracy once budget crosses \$100
AlphaProof / AlphaGeometry	50 200 token steps	RL-based reward tweaks inference-time policy to maintain correctness under limited latency

9.2 Benchmark 仪表与预算分层

Chen 建议为每个 benchmark 都建立一个 “budget bin” 仪表板：把 token/cost 预算分为 Conservative、Balanced、Aggressive 三档，分别对应 ‘20 80 token’、‘80 300 token’、‘>300 token’ 的情况。每个 bin 都要记录 accuracy、stepwise verifier margin、token per vote，以便在运行时快速定位 “当前状态是哪个 bin”。这种分层可帮助团队在对外发布 benchmark 结果时直接说 “在 Balanced bin 下，我们的 accuracy 87.5% 但 latency 3.4s”。

表 5: Benchmark 仪表示例

Bin	典型配置	监控指标
Conservative	≤ 80 token, ≤ 2 iterations	Accuracy, sampling diversity, latency
Balanced	80 300 token, 2 3 iterations	Verifier margin, branching ratio, token per candidate
Aggressive	> 300 token, 3+ iterations	Score per token, trace iterations, SLO breach rate

Benchmark 仪表的落地建议

1. 为每个 bin 配置一个 ‘alert level’，当 latency $>$ target 或 verifier margin $<$ threshold 时切换 bin；2. 把 bin 的 state push 到 dashboard，让运营团队可以快速阅读 “今天的 benchmark 运行在哪一档”；3. 结合 bin 内的 score-per-token 监控，及时发现 token budget 失控的 early warning。

9.3 开放研究问题

- 如何在低 latency 场景下用有限 compute 拓展 search depth 与 iteration depth?
- 能否设计一个 “budget-aware verifier” 实时报告 token/candidate tradeoff，而不要等到 inference

完成？

- Trace feedback 在多模态/图像 + 文本 reasoning 中的通用性如何保障？
- 在真实部署 pipeline 中如何把 sampling diversity、stepwise scaling metric、trace drift 等指标串成可执行的 SLO？
- 推理时 pipeline 能否用 learning-to-rank 的方式自适应调整 branching ratio 与 iteration depth，而不是人工设定阈值？
- 对于 adversarial prompt，verifier margin 如何与 prompt-editing loop 结合，以免错误推理被 feedback 强化？
- 多模态 reasoning 下，token 预算与 vision token length 的对应关系还有哪些不确定性？

这些问题在 01:19:00 之后的 Q&A 被反复提及，Chen 建议通过 “metric-first hypothesis” 方式探索：先提出核心 metric（如 ‘score per token’）再迭代调整 pipeline，而非凭直觉扩大 token budget。

研究方向联想

开放问题可以从 budgeting instrumentation、verifier robustness 以及 trace 科学三个角度切入。Chen 建议把 inference-time logs (sampling count/delay/score) 写入 dashboard，为未来自动调整策略提供基础数据。

9.4 本章小结

通过对 Arc AGI、O3 以及 DeepMind 系列的 budget/accuracy 曲线解读，我们看到 inference-time compute 既是工程参数也是研究方向的触发器。未来需要在 low-latency 下维持 high accuracy，并把 trace/verifier/gating 指标织入部署 pipeline。

10 部署与观测

10.1 监控 inference-time 指标

Chen 把 inference-time pipeline 看成一个 “黑盒” —— 输入 prompt/block 之后，依次走过 sampling、search、trace，每一段都应该输出 metrics。典型的 metrics 包含 sampling depth、branching ratio、verifier score distribution、trace iteration count 与 latency。建议在 dashboard 上按 5 分钟一组绘制这些曲线，真实系统中一旦 sampling depth 突然下降或 trace iteration 突增，就要触发调查。

Observability 工具链

1. Sampling log: 记录 prompt + generated candidates + token count; 2. Search log: 记录 beam/tree status (depth, score, branch id); 3. Trace log: 输出每轮 self-correction 是否有人类命名的 error pattern; 4. Alert rules: 设定 collector，在 sample 多样性低于 baseline 或 trace drift 超过 threshold 时告警。

10.2 Budget SLO 与警报

在部署 inference-time compute pipeline 时, 常用 SLO 表格列出 token budget、search depth 与 iteration depth 的上限, 并配套 “当 average token per query $> \text{budget} \times 1.1$ ” 或 “search branching > 30 ”

表 6: Inference-time SLO 示例

Metric	Target	Alert condition
Token budget per query	≤ 400 tokens	5-minute moving average > 440 tokens
Sampling diversity	80% unique candidates	3 consecutive minutes diversity $< 60\%$
Iteration depth	≤ 2 feedback rounds	iteration count > 3 for top 1% queries
Verifier score margin	≥ 0.15	margin drop > 0.05 for 100 queries

警报调优建议

不要让每一次 trace drift 都弹警报, 否则会造成 alarm fatigue。Chen 的经验是把误报阈值设得稍高, 真正有问题时再调度人工排查, 并辅以 “replay” 功能让团队复盘 sampling log。

10.3 本章小结

部署 inference-time pipeline 时必须把 observability、SLO、警报与 log replay 联动, 才能在 production 层保持 accuracy 和 reliability, 同时把 budget 运行在可控范围之内。

11 实践示例与指令模板

11.1 Prompt Template 演练

一个理想的 inference-time prompt 包含 context、task difficulty、example CoT、instrumentation hints 以及 final instruction。例如下面的模板在多步骤 reasoning 中较常使用:

```
1 System: You are a high-precision reasoning assistant.
2
3 Input context: {problem description}
4 Difficulty: {mark as "hard"/"critical"}
5 Examples:
6   - Example 1 CoT: ...
7   - Example 2 CoT: ...
8 Instruction: Think step-by-step. If uncertain, enumerate two next steps and sample multiple candidates.
```

Listing 1: Inference-time prompt skeleton

提示模板的落地技巧

把 difficulty 作为 metadata 绑定在 prompt 里，便于 sampling policy 自动调整 temperature 与 token limit；用例示例表达 “wrong → right” 对比，提醒模型要把 reasoning 拓展到深度。

11.2 Self-Consistency Orchestration

在 sampling 阶段，先用 high-temperature 不断生成 20 40 个 responses；在 search 阶段，把这些 responses 看作 seed，开启 beam/tree search 进一步延展 promising branches；在 iteration 阶段，运行 trace feedback 和 verifier 评分，再基于 majority vote/consistency 选出最终 answer。

Orchestration checklist

1. Sampling: 记录 candidate count + diversity;
2. Search: track branch depth, stuck rate;
3. Iteration: log trace iteration count, verification score, self-correction delta.

11.3 本章小结

实践示例帮助把 abstract 的 inference-time 理念具体化：我们可以借助 prompt skeleton、sampling/search/iteration 三阶段 orchestrations，以及 instrumentation checklist 来把 pipeline 写成可复用的 template。

12 风险与治理

12.1 风险识别

常见的 risk 包括 budget 失控 (token budget 突然突破)、verifier drift (score margin 下降)、trace drift (self-correction 不再提升 accuracy) 以及 sampling collapse (diversity 下降)。每当这些风险出现时，Chen 建议立即回滚 sampling policy (温度、nucleus)，并且用 “trace replay” 复现决策过程。

风险信号与初期应对

1. Token budget > 110% of SLO → reduce search width;
2. Verifier margin drop > 0.05 → rerun sampling with new prompts;
3. Trace iteration spike → throttle iteration depth and inspect trace log.

12.2 治理与演练

在治理层面，需建立 inference-time runbook，包括 budget gatekeeper、trace simulator、post-mortem template。当 pipeline 异常时，可以用 runbook 迅速切换到 conservative mode (降低 token budget，限定 search depth) 并把异常 log 录入 backlog，用于 future prompt iteration。

表 7: 治理演练所需资产

Asset	Purpose	Trigger
Budget gatekeeper dashboard	显示 token/search/iteration 用量, 支持 manual override	Budget SLO breach
Trace simulator	Replay candidate + verifier steps for debugging	Consistency drop or new failure mode
Post-mortem template	Capture root cause, fix, and regression test plan	Any production inference failure

12.3 本章小结

有了清晰的风险识别、runbook 与治理资产, Inference-time pipeline 才能在多轮迭代中保持稳定, 保障 prompt/search/trace 各环节按预算、按指标运行。

13 案例分析与下一步

13.1 Arc AGI Puzzle Walkthrough

在 Arc AGI 这条 benchmark 曲线上, 我们可以用以下步骤驱动 inference loop: 1) Prompt 里提前声明这个 puzzle 是 high-difficulty; 2) Sampling 阶段生成 30 40 个 candidate response; 3) Search 阶段把这些 candidate 当作 tree seed, 用 stepwise scorer 计算 promisingness 并且 intro probability threshold; 4) Iteration 阶段运行 trace feedback, 自我检查每条 response 的 reasoning chain, 并借助 verifier margin 提示多数票选择。整个流程也会记录 every candidate's token cost 与 latency, 供 future tuning。

Arc AGI Pipeline 观察

把 Arc AGI 视为全流程演练场: prompt → sampling → search → trace → voting, 每一环都需要 instrumentation; 如果 token cost 突然翻倍, 看 sampling diversity 与 verifier margin 是否崩溃, 然后用 runbook 回滚到 conservative mode。

13.2 高价值任务的推理时部署

对于像 AlphaProof 这种竞争性 Code/Math task, pipeline 里通常还会加入 extra verification: 1) Run deterministic verifier (e.g. proof checker) as soon as the candidate completes; 2) If verification fails, reroute to alternative candidate with similar margin; 3) Log the mismatch and feed trace back to sampling controller for prompt iteration。这样的流程使 inference-time compute 成为一条 closed-loop control rather than a one-shot guess。

高价值部署要点

1. 把 deterministic verifier 作为 fallback, 避免 majority vote 把 wrong candidate 当成正确答案;
2. 用 trace replay 复现每一次 verification failure, 以便 prompt 出新的 examples;
3. 把 close-loop pipeline 的 metrics (token cost, verifier success, trace iteration) 记录在 performance dashboard。

13.3 本章小结

案例分析展示了如何把推理时 compute pipeline 实际落地到 Arc AGI 与 AlphaProof 类任务: 先确定 prompt 级别, 再让 sampling/search/trace 组成控制钮, 最后把 verification 反馈写入 iteration loop 以持续改进。

14 总结与延伸

本讲围绕 inference-time compute 展开: 先让提示释放 token, 再用 sampling/search 扩展 solution space, 最后靠 trace + self-correction 修正结果。三段可以按需组合, 在高质量 reasoning 任务中显著提升 accuracy 与 reliability。

我们还梳理了 compute scheduling (三角调度)、self-consistency/search 的协同机制、stepwise verifier 与 Tree-of-Thought 的控制 loop, 以及 benchmark 的数据解读, 形成一张从 prompt 到 deployment 的完整路线图。

确认落地的要点

1. 在工程中持续追踪 sampling、search、trace 三类指标;
2. 把 benchmark 曲线 (例如 Arc AGI) 作为 budget gating 的参考;
3. 把 deployment SLO、alert、budget instrumentation 写入 CI/CD pipeline, 确保 inference-time compute 不会超出可控范围。

表 8: 推理时计算的核心套路

策略	关键收益	典型代价
扩展 CoT 预算	让模型在难题上逼近 human-level accuracy	更多 token、更多 latency、prompt 设计更敏感
多候选采样与 self-consistency	从 mistakes 中恢复, 适用于没有 verifier 的场景	每条 candidate 都要 compute, 需 voter pool 与一致性表征
Tree-of-Thought 与 step verifier	早期淘汰 bad paths、与 verifier 配合提升可解释性	branching 数量激增, 需要设计 quality function
Trace feedback + self-correction	Line-by-line 评分可逐步提升 accuracy	无 oracle 会导致 feedback drift、需要 prompt guard rails

14.1 本章小结

推理时技术需把提示、search、trace 合成 pipeline, 通过预算调度与指标回路让每个模块在不同阶段发挥最大效益。

14.2 进一步阅读

- Wang 等人《Self-Consistency improves chain of thought reasoning》阐释了 sampling + majority vote 的 scaling 效果。
- Zhou 等人《Tree-of-Thought》提出通过树状探索把 search 与 step evaluation 结合。
- 《Trace-Enhanced LLM Safety》等 trace feedback 论文说明 natural language trace 如何辅助 verifier。
- DeepMind 的 AlphaProof/AlphaGeometry/AlphaCode 系列展示了 inference-time RL + search + trace 的组合在复杂 reasoning 任务中的成果。